

Task #1 from the lab (works)

```
In [22]: import cv2
# Load the image in grayscale
img = cv2.imread('images/Parrot.png', cv2.IMREAD_GRAYSCALE)
# Set the threshold values
threshold_values = [0, 50, 100, 150, 200]
# Apply thresholding with different threshold values
for threshold_value in threshold_values:
    ret, thresh = cv2.threshold(img, threshold_value, 255,
                                cv2.THRESH_BINARY)
    cv2.imshow(f'Thresholded Image ({threshold_value})',
               thresh)
# Display the original image
cv2.imshow('Original Image', img)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Task #2 from the lab

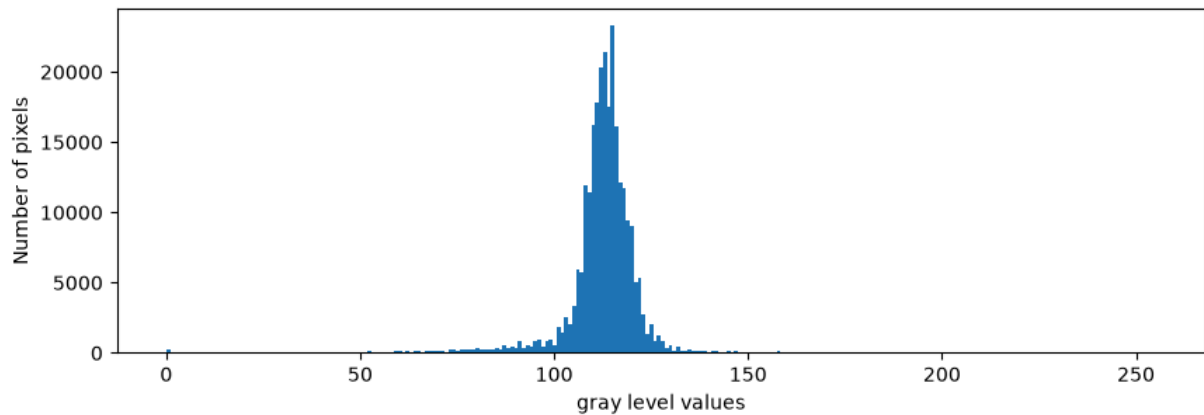
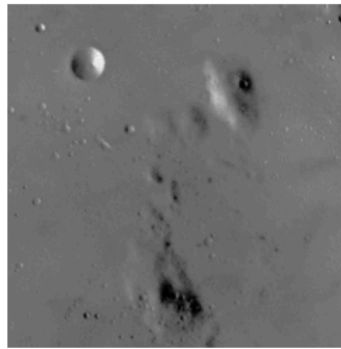
```
In [23]: import matplotlib.pyplot as plt
import numpy as np
from skimage import data, img_as_float
from skimage import exposure
```

```
In [24]: img = data.moon()
```

```
In [25]: # Contrast stretching
p2, p98 = np.percentile(img, (2, 98))
img_rescale = exposure.rescale_intensity(img, in_range=(p2, p98))
```

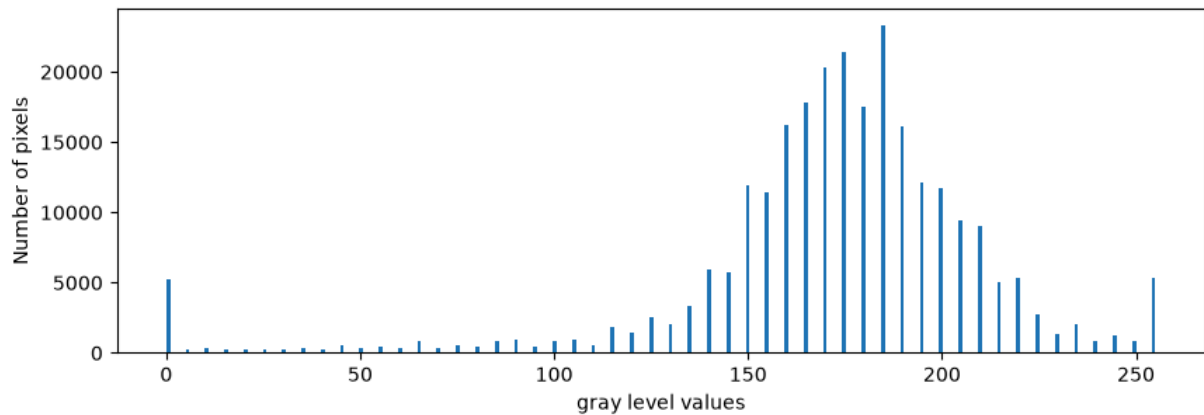
```
In [26]: fig = plt.figure(figsize=(10, 7))
fig.add_subplot(2, 1, 1)
plt.imshow(img, cmap = 'gray')
plt.axis('off')
plt.title('Original image')
fig.add_subplot(2, 1, 2)
plt.hist(img.flat, bins = 256, range=(0, 255))
plt.xlabel('gray level values')
plt.ylabel('Number of pixels')
plt.show()
```

Original image



```
In [27]: fig = plt.figure(figsize=(10, 7))
fig.add_subplot(2, 1, 1)
plt.imshow(img_rescale, cmap = 'gray')
plt.axis('off')
plt.title('Contrast stretch')
fig.add_subplot(2, 1, 2)
plt.hist(img_rescale.flat, bins = 256, range=(0, 255))
plt.xlabel('gray level values')
plt.ylabel('Number of pixels')
plt.show()
```

Contrast stretch



Assessments

Task 1

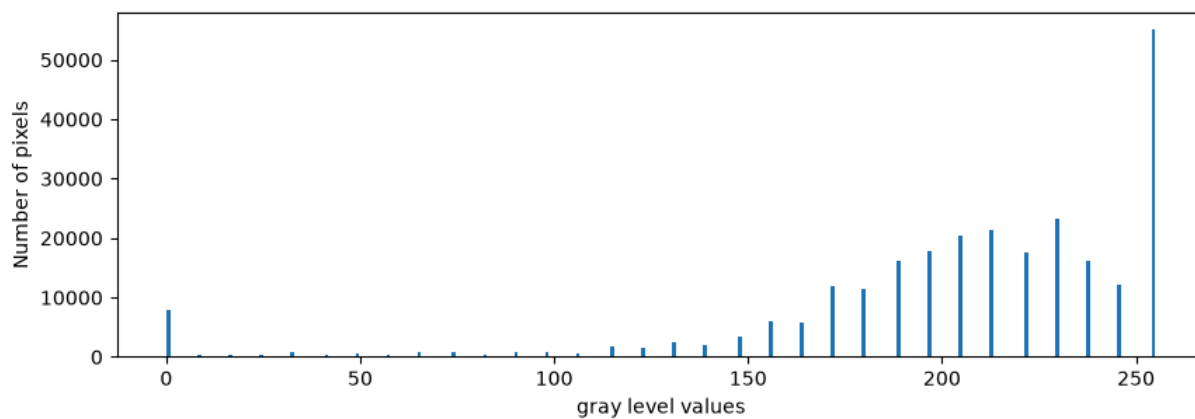
```
In [28]: p3, p80 = np.percentile(img, (3, 80))
img_rescale = exposure.rescale_intensity(img, in_range=(p3, p80))

fig = plt.figure(figsize=(10, 7))

fig.add_subplot(2, 1, 1)
plt.imshow(img_rescale, cmap = 'gray')
plt.axis('off')
plt.title('Contrast stretch (3rd - 80th percentile)')

fig.add_subplot(2, 1, 2)
plt.hist(img_rescale.flat, bins = 256, range=(0, 255))
plt.xlabel('gray level values')
plt.ylabel('Number of pixels')
plt.show()
```

Contrast stretch (3rd - 80th percentile)



Task 2

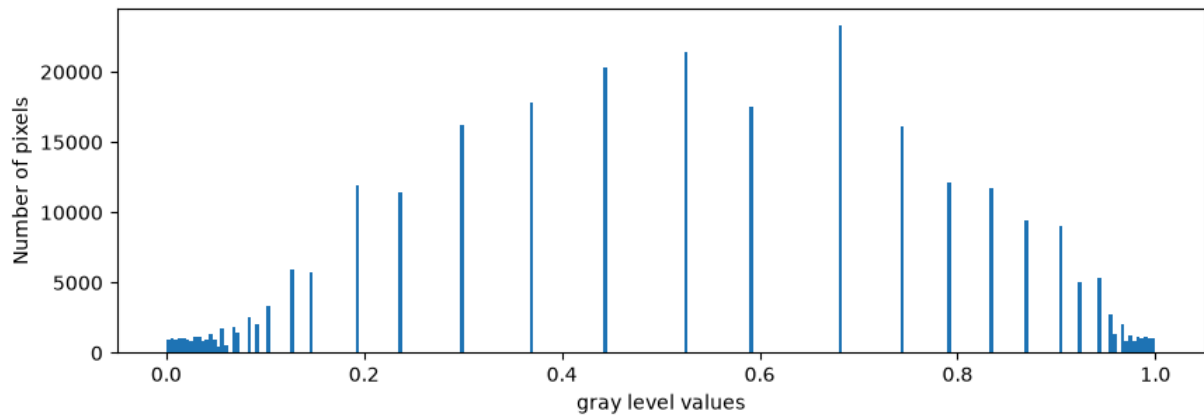
```
In [29]: img_eq = exposure.equalize_hist(img)

fig = plt.figure(figsize=(10, 7))

fig.add_subplot(2, 1, 1)
plt.imshow(img_eq, cmap = 'gray')
plt.axis('off')
plt.title('Histogram equalization')

fig.add_subplot(2, 1, 2)
plt.hist(img_eq.flat, bins = 256, range=(0, 1))
plt.xlabel('gray level values')
plt.ylabel('Number of pixels')
plt.show()
```

Histogram equalization



Task 3

```
In [30]: reference = data.rocket()
         image = data.chelsea()

         matched = exposure.match_histograms(image, reference, channel_axis=-1)
```

```
In [31]: fig = plt.figure(figsize=(15, 5))

         fig.add_subplot(1, 3, 1)
         plt.imshow(image)
         plt.axis('off')
         plt.title('Source (Chelsea)')

         fig.add_subplot(1, 3, 2)
         plt.imshow(reference)
         plt.axis('off')
         plt.title('Reference (Rocket)')

         fig.add_subplot(1, 3, 3)
         plt.imshow(matched)
         plt.axis('off')
         plt.title('Matched')
         plt.show()
```

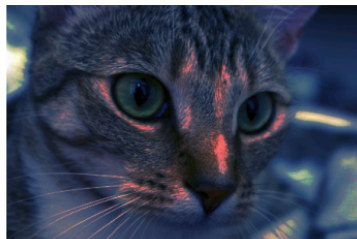
Source (Chelsea)



Reference (Rocket)



Matched



In []: